

Street Directionality Problem - Report

Assignment 2: Linear Programming

Tommaso Bergonzoni

May 2026

1 Introduction

The goal of this project is to solve the Street Directionality Problem by maximizing the number of two-way streets converted into one-way streets, while ensuring that the travel time between any two reachable crossings does not increase by more than p .

2 General Strategy

The strategy consists of first computing the initial shortest paths (Floyd-Warshall Algorithm) between all crossings to obtain base data for reachability and travel times.

Following this preprocessing, the problem is solved using Integer Linear Programming via IBM ILOG CPLEX.

To be more precise, the sequential steps of Floyd-Warshall are directly embedded into the mathematical model as linear constraints. The exact routing topology is preserved using continuous variables, binary decision variables and a Big-M formulation. The model guarantees that any travel time increase remains within the $p\%$ threshold, keeping the original reachability between each node.

3 Implementation Details

3.1 Variables

From the input file, the code extracts the following data:

- **n**: Number of crossings.
- **t_matrix_{ij}**: The initial time matrix, where **t_matrix_{ij}** represents the time to go from crossing i to j directly.
- **p**: The threshold percentage for the maximum allowed increase in travel time.

The algorithm defines the following variables:

- **d_init**: An $n \times n$ matrix representing the minimum time distance between any two crossings before any changes are made.
- **X_{i,j}**: A 1D array of binary decision variables. The variable X_{ij} is equal to 1 if traffic is permitted to flow directly from crossing i to crossing j , and 0 otherwise.
- **D_{k,i,j}**: A 1D array of continuous decision variables. This structure embeds the iterative states of the Floyd-Warshall algorithm directly inside the LP solver, where $D_{k,i,j}$ represents the minimum travel time from i to j restricted to using intermediate crossings from the set $\{0, 1, \dots, k-1\}$.

- c_{ij} : A dynamic array of binary decision variables allocated exclusively for existing two-way streets ($t_{ij} > 0$ and $t_{ji} > 0$). This variable acts as a transformation flag, taking the value 1 if the two-way street $\{i, j\}$ is converted into a one-way street, and 0 if it remains open in both directions.

3.1.1 Objective Function and Mathematical Model

The goal is to maximize the total number of two-way streets that are converted into one-way streets while respecting all constraints. Mathematically, let B be the set of pairs (i, j) such that $i < j$ and a bidirectional street exists between crossing i and crossing j in the initial network topology.

Binary variables $c_{i,j}$ indicate if one of the ways can be closed (1) or not (0).

The objective function to be maximized is defined as follows:

$$\max \sum_{(i,j) \in B} c_{ij}$$

3.1.2 Initial Minimum Distances

The initial minimum distances are computed during the preprocessing using the Floyd-Warshall algorithm. This approach was chosen simply to maintain consistency with the previous assignment, allowing the reuse of the same matrix-based shortest path logic.

3.1.3 Constraints

To ensure the validity of the solutions, the following constraints were implemented:

- *Reachability*: For every pair of crossings (i, j) that were reachable in the original graph, they still need to be reachable after the conversion of any two-way streets.
- *Travel Time*: For every pair of reachable crossings (i, j) , the new shortest path distance $D_{n,i,j}$ must satisfy the condition: $D_{n,i,j} \times 100.0 \leq d_{init,i,j} \times (100 + p)$.
- *Orientation*: Each two-way street can either remain bidirectional or be converted to a one-way street in only one of the two possible directions, as determined by the decision variables X_{ij} and X_{ji} .

The value of each indicator c_{ij} is dynamically constrained by the decision variables X_{ij} and X_{ji} using the following system of linear inequalities:

$$\begin{aligned} c_{ij} &\leq 2 - X_{ij} - X_{ji} \\ c_{ij} &\geq X_{ij} - X_{ji} \\ c_{ij} &\geq X_{ji} - X_{ij} \end{aligned}$$

This last system represents a simple implementation of the logic operator XOR in a mathematical way.

4 Results

I changed the model to Integer Linear Programming (ILP) using the CPLEX solver. The performance is much better now compared to the previous constraint programming version. This time, I successfully solved all the problem instances provided in the suite.

The execution time was reduced a lot. The solver finds the optimal solution almost instantly for most of the test cases. Every output file satisfies the street orientation rules and the maximum travel time threshold p . All solutions passed the validation checks.